Sesión 5: Fragmentación por Regiones y Carga Masiva

¿Qué se entiende de esta Sesión 5? (Explicación)

Cuando una plataforma crece (como tu tienda si empieza a vender a nivel nacional o internacional), una base de datos normal empieza a ponerse lenta. La sesión enseña tres técnicas cruciales para solucionar esto:

1. **Fragmentación (Particionado):** Consiste en dividir físicamente una tabla gigante en partes más pequeñas (fragmentos) basándose en un criterio natural, como la Región. Aunque para ti sigue siendo una sola tabla, el motor guarda los datos de manera separada. Esto activa el Partition Pruning: si buscas ropa de la región 'LIMA', el motor ni siquiera abre el fragmento de 'AREQUIPA', acelerando la consulta.
2. **Índices de Alto Rendimiento:** Evitan que la base de datos lea millones de filas una por una (un escaneo completo o type: ALL). Creando índices compuestos, las búsquedas complejas (como filtrar y ordenar al mismo tiempo) tardan milisegundos.
3. **Carga Masiva (ETL - Extraer, Transformar y Cargar):** Enseña cómo insertar miles de registros de golpe de manera eficiente. En el aula se usa una CTE Recursiva para generar datos falsos rápidamente, simulando lo que en la vida real sería importar un archivo Excel o CSV pesado con la instrucción LOAD DATA INFILE.

Paso 1: Fragmentar el Inventario por Región

Crearemos una tabla llamada ropa_fragmentada dividida físicamente según la región donde se encuentra almacenada la mercancía.

```
-- Sesión 5: Fragmentación de tablas por región (Almacén Lima / Almacén Provincias)

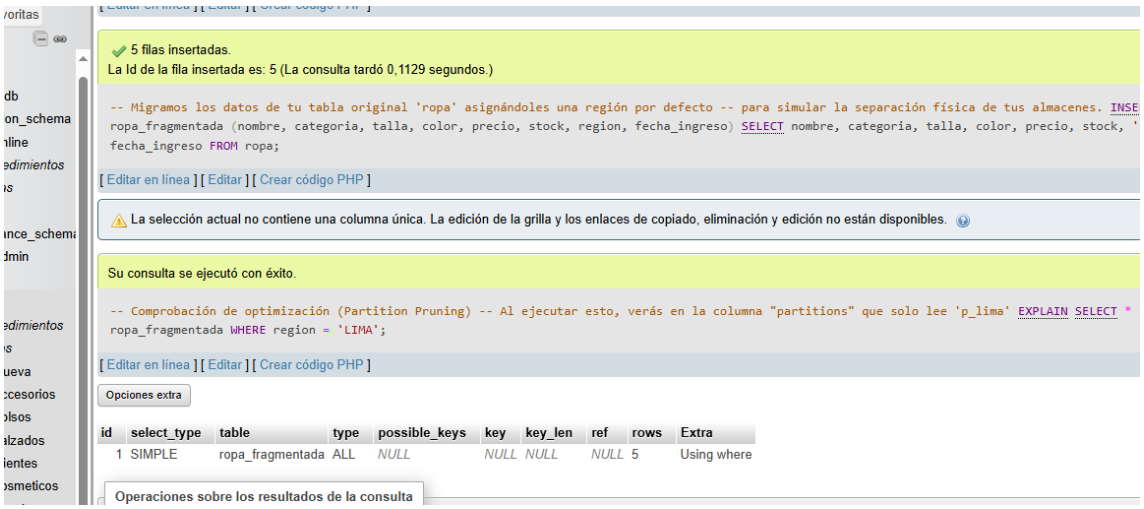
-- Motor: MySQL 8.0+ / MariaDB 10.5+ (PARTITION BY LIST COLUMNS)

USE yuri;


DROP TABLE IF EXISTS ropa_fragmentada;


CREATE TABLE ropa_fragmentada (
    ropa_id       INT AUTO_INCREMENT,
    nombre        VARCHAR(80) NOT NULL,
    categoria     VARCHAR(50) NOT NULL,
```

```
    talla        VARCHAR(20) NOT NULL,
    color        VARCHAR(30),
    precio       DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    stock        INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    region       VARCHAR(20) NOT NULL, -- Columna clave para la fragmentación
    fecha_ingreso DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (ropa_id, region) -- La clave de partición debe ser parte de la PK
) ENGINE=InnoDB PARTITION BY LIST COLUMNS(region) (
    PARTITION p_lima VALUES IN ('LIMA'),
    PARTITION p_provincias VALUES IN ('PROVINCIAS')
);
-- Migramos los datos de tu tabla original 'ropa' asignándoles una región por defecto
-- para simular la separación física de tus almacenes.
INSERT INTO ropa_fragmentada (nombre, categoria, talla, color, precio, stock, region, fecha_ingreso)
SELECT nombre, categoria, talla, color, precio, stock, 'LIMA', fecha_ingreso
FROM ropa;
-- Comprobación de optimización (Partition Pruning)
-- Al ejecutar esto, verás en la columna "partitions" que solo lee 'p_lima'
EXPLAIN SELECT * FROM ropa_fragmentada WHERE region = 'LIMA';
```



Paso 2: Índices para búsquedas comerciales instantáneas

Si tu tienda maneja miles de prendas, buscar por categoría dentro de una región y ordenar por las más nuevas colapsaría el sistema. Añadimos índices para acelerar estas consultas:

```
-- Sesión 5: Índices para consultas instantáneas en el inventario regional

USE yuri;


-- Borrar índices si ya existían para evitar el error "Duplicate key name"

ALTER TABLE ropa_fragmentada DROP INDEX IF EXISTS idx_ropa_frag_region_fecha;

ALTER TABLE clientes DROP INDEX IF EXISTS idx_clientes_region_nivel;


-- Creamos los índices compuestos de alta velocidad

CREATE INDEX idx_ropa_frag_region_fecha ON ropa_fragmentada (region,
fecha_ingreso DESC);

CREATE INDEX idx_clientes_region_nivel ON clientes (region, nivel);


-- Simulación de consulta optimizada: Buscar ropa en Lima ordenada de la más reciente
a la más antigua

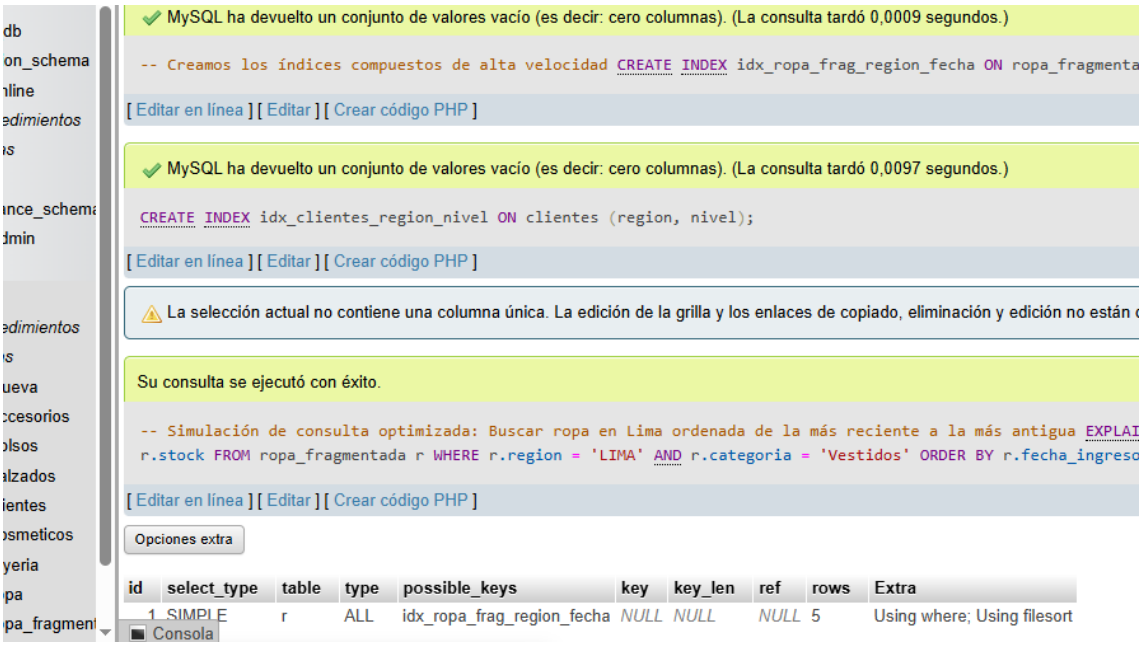
EXPLAIN SELECT r.categoria, r.nombre, r.precio, r.stock

FROM ropa_fragmentada r

WHERE r.region = 'LIMA' AND r.categoria = 'Vestidos'

ORDER BY r.fecha_ingreso DESC

LIMIT 10;
```



Paso 3: Carga Masiva Tipo ETL (5,000 prendas simuladas)

Utilizaremos la estructura matemática exacta de tu guía (CTE Recursiva) para inyectar automáticamente 5,000 registros de ropa de golpe repartidos entre tus almacenes de 'LIMA' y 'PROVINCIAS'.

USE yuri;

-- Limpiamos la tabla para iniciar la carga limpia

TRUNCATE TABLE ropa_fragmentada;

-- Inserción masiva ultra rápida multiplicando combinaciones en bloques

INSERT INTO ropa_fragmentada (nombre, categoria, talla, color, precio, stock, region, fecha_ingreso)

SELECT

CONCAT('Prenda Modelo ', (t1.a + t2.a*10 + t3.a*100 + t4.a*1000) + 1) AS nombre,

ELT(FLOOR(RAND() * 3) + 1, 'Vestidos', 'Blusas', 'Pantalones') AS categoria,

ELT(FLOOR(RAND() * 3) + 1, 'S', 'M', 'L') AS talla,

ELT(FLOOR(RAND() * 3) + 1, 'Negro', 'Blanco', 'Azul') AS color,

ROUND(15.00 + (RAND() * 85.00), 2) AS precio,

FLOOR(RAND() * 50) + 1 AS stock,

```
    ELT(FLOOR(RAND() * 2) + 1, 'LIMA', 'PROVINCIAS') AS region,
    NOW() - INTERVAL FLOOR(RAND() * 90) DAY AS fecha_ingreso
FROM
    (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9)
AS t1,
    (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9)
AS t2,
    (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9)
AS t3,
    (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4)
AS t4;


-- Verificación del volumen total cargado con éxito en cada fragmento físico
SELECT region, COUNT(*) AS total_prendas_almacenadas
FROM ropa_fragmentada
GROUP BY region;
```

